UNIT III: Constructors, Destructors and Operator Overloading

Question Bank: Debugging & Pseudo-Code Formulation (Questions Only)

---

1. Syllabus & Topic Scope

- **Constructors**: Definition, characteristics, default constructors, parameterized constructors, dynamic initialization of objects.
- **Constructor Overloading**: Multiple constructors with differing signatures, delegating responsibilities.
- **Copy Constructors**: Syntax `ClassName(const ClassName &other)`, compiler-synthesized copy constructor, shallow copy vs. deep copy hazards.
- **Dynamic Constructors**: Allocating heap memory dynamically inside constructor bodies using `new`.
- **Destructors**: Syntax `~ClassName()`, lifetime termination, cleanup of dynamically allocated resources (`delete` / `delete[]`), invocation order (reverse of constructor order).
- **Defining Operator Overloading**: Syntax and semantics of `operator` keyword, member operator functions vs. friend operator functions.
- **Overloading Unary & Binary Operators**: Overloading unary operators (prefix `++`, postfix `++(int)`, unary `-`), binary arithmetic (`+`, `-`, `*`), relational (`==`, `!=`, `<`, `>`).
- **Stream Insertion (`<<`) & Extraction (`>>`) Operators**: Implementing stream I/O operators as non-member/friend functions.
- **Rules for Overloading Operators**: Un-overloadable operators (`.`, `.*`, `::`, `?:`, `sizeof`), preserving precedence, associativity, and arity, prohibition of inventing new operators, requirement of at least one user-defined operand.

---

2. Assessment Instructions & Rules ("How to Do & What to Do")

A. Instructions for Debugging Questions

When debugging constructors, destructors, and operator overloading implementations:

1. **Inspect Copy Constructor Signatures**:
   - Verify that the parameter is passed **by reference** (`const ClassName &obj`). Passing by value (`ClassName obj`) causes an infinite compilation recursion loop.
2. **Examine Heap Resource Management (Rule of Three)**:
   - If a class uses dynamic allocation (`new` / `new[]`) in its constructor and frees it in a destructor, check whether a custom **Deep Copy Constructor** is implemented. Missing deep copy results in **Shallow Copy Pointer Aliasing** and runtime **Double-Free Heap Corruption**.
3. **Verify Operator Placement & Signatures**:

- **Stream Operators (`<<`, `>>`)**: Cannot be member functions when `std::cout` or `std::cin` appears on the left-hand side. They must be declared as **non-member or friend functions** returning `std::ostream&` / `std::istream&`.
- **Prefix vs. Postfix Increment**:
  - Prefix (`++obj`): takes no arguments, modifies object, returns reference `Type&`.
  - Postfix (`obj++`): takes dummy `int` token, captures old state, increments object, returns copy `Type`.

4. Enforce Language Overloading Constraints:

- Verify that non-overloadable operators (`.`, `.*`, `::`, `?:`, `sizeof`) are never overloaded.
- Ensure at least one operand is a user-defined class or enum type. Built-in types cannot be overloaded.

5. Required Response Format:

- Identify line numbers and classify the bug.
- Provide the theoretical root-cause analysis referencing C++ memory models or grammar rules.
- Provide the clean, corrected C++ code.
- Show the dry-run output.

B. Instructions for Pseudo-Code Writing Questions

1. Lifecycle Modeling:

- Explicitly denote `CONSTRUCTOR(...)`, `COPY CONSTRUCTOR(REF other)`, and `DESTRUCTOR()`.
- Model explicit heap allocation (`ALLOCATE ARRAY[...]`) and deallocation (`DEALLOCATE pointer`).

2. Operator Notation:

- Use formal operator signatures: `OPERATOR +(REF other AS Type) RETURNS Type` or `FRIEND OPERATOR <<(REF stream, REF obj)`.

3. Deep Copy Logic:

- Demonstrate the allocation of a fresh memory buffer and the item-by-item copy loop inside copy constructors.

---

3. Section A: Debugging Questions (5 Problems)

Question 1: Copy Constructor Pass-by-Value Infinite Recursion Trap

Topic: Copy Constructors, Parameter Passing Mechanisms

Problem Statement

A student designs a custom dynamic string class `DynamicString` that allocates character buffers on the heap. The student attempts to define a copy constructor to enable initialization of one string from another. However, the C++ compiler aborts compilation with a fatal error on the copy constructor signature.

Buggy Code

```
#include <iostream>
#include <cstring>

class DynamicString {
private:
    char *buffer;
    int length;

public:
    // Parameterized Constructor
    DynamicString(const char *str) {
        length = std::strlen(str);
        buffer = new char[length + 1];
        std::strcpy(buffer, str);
    }

    // Line 17: Copy Constructor defined with Pass-by-Value!
    DynamicString(DynamicString source) {
        length = source.length;
        buffer = new char[length + 1];
        std::strcpy(buffer, source.buffer);
    }

    ~DynamicString() {
        delete[] buffer;
    }

    void print() const {
        std::cout << buffer << std::endl;
    }
};

int main() {
    DynamicString s1("Antigravity C++");
    DynamicString s2 = s1; // Intended copy construction
    s2.print();
    return 0;
}
```

Student Tasks

1. State the exact compiler error generated by **Line 17**.

2. Explain the theoretical mechanism: Why does passing `source` by value in a copy constructor lead to infinite recursive calls at compile time?

3. Provide the corrected copy constructor signature and explain the importance of adding the `const` qualifier and the pass-by-reference operator `&`.

4. Provide the complete compilable code snippet.

Question 2: Shallow Copy & Double-Free Runtime Crash

Topic: Dynamic Constructors, Destructors, Shallow vs. Deep Copy

Problem Statement

An audio processing class allocates dynamic heap storage for integer audio samples in its constructor and deletes the memory in its destructor. However, the author omits a user-defined copy constructor. When an object is passed by value to an analysis function, the program compiles cleanly but crashes violently during runtime with a heap corruption abort.

Buggy Code

```
#include <iostream>

class AudioBuffer {
public:
    int *samples;
    int capacity;

    AudioBuffer(int cap) : capacity(cap) {
        samples = new int[capacity];
        for (int i = 0; i < capacity; ++i) {
            samples[i] = i * 10;
        }
        std::cout << "Buffer allocated at: " << samples << std::endl;
    }

    // Programmer omitted copy constructor!

    ~AudioBuffer() {
        std::cout << "Deallocating buffer at: " << samples << std::endl;
        delete[] samples; // Crash occurs here during execution!
    }
};

void processAudio(AudioBuffer buf) { // Passed by value!
    std::cout << "Processing sample 0: " << buf.samples[0] << std::endl;
}

int main() {
    AudioBuffer track(5);
    processAudio(track);
    std::cout << "Back in main, sample 0: " << track.samples[0] << std::endl;
    return 0;
}
```

Student Tasks

1. Trace the memory address of `samples` before, during, and after `processAudio(track)` executes. What does the compiler-synthesized default copy constructor do to raw pointer members?

2. Explain why the program crashes with `free(): double free detected` when `main()` exits.

3. Write a user-defined **Deep Copy Constructor** that allocates fresh memory for the copied object and copies the elements individually.

4. Show the verified terminal output showing both distinct allocations and safe deallocations.

Question 3: Stream Insertion Operator `<<` Member Function Violation

Topic: Defining Operator Overloading, Member vs. Non-Member/Friend

Problem Statement

A student implements a `Fraction` class and attempts to overload the stream insertion operator `<<` as a member function so that fractions can be printed using `std::cout << frac;`. The compiler rejects the expression in `main()`, but bizarrely permits the reverse expression `frac << std::cout;`.

Buggy Code

```
#include <iostream>

class Fraction {
private:
    int numerator;
    int denominator;

public:
    Fraction(int n, int d) : numerator(n), denominator(d) {}

    // Line 11: Declaring operator<< as a member function
    std::ostream& operator<<(std::ostream &os) {
        os << numerator << "/" << denominator;
        return os;
    }
};

int main() {
    Fraction f1(3, 4);

    // Line 20: Natural stream syntax fails to compile!
    std::cout << f1 << std::endl;

    // Line 23: Compiles, but inverts standard semantics
    f1 << std::cout;

    return 0;
}
```

Student Tasks

1. Explain how the compiler translates binary member operator calls (`left @ right` $\rightarrow$ `left.operator@(right)`). Why does this translation cause `std::cout << f1` to fail compilation when `operator<<` is a member of `Fraction`?

2. Explain why `f1 << std::cout` compiles and why this syntax is completely unacceptable in production code.

3. Rewrite `operator<<` as a non-member `friend` function taking `std::ostream&` and `const Fraction&` as parameters.

4. Show the corrected code and resulting output.

Question 4: Prefix vs. Postfix Increment Signature Inversion

Topic: Overloading Unary Operators (Prefix `++`, Postfix `++(int)`)

Problem Statement

A `StepCounter` class overloads both prefix and postfix increment operators. The author mixes up the canonical signatures, parameter lists, and return semantics, resulting in logic failures when chaining prefix increments or assigning postfix snapshots.

Buggy Code

```
#include <iostream>

class StepCounter {
private:
    int steps;

public:
    StepCounter(int s = 0) : steps(s) {}

    // Line 10: Intended as POSTFIX, but missing dummy int parameter
    StepCounter operator++() {
        StepCounter temp = *this;
        steps++;
        return temp;
    }

    // Line 17: Intended as PREFIX, but includes dummy int parameter
    StepCounter& operator++(int) {
        steps++;
        return *this;
    }

    void display() const {
        std::cout << "Steps: " << steps << std::endl;
    }
};

int main() {
    StepCounter c1(10);

    // Developer expects prefix (++c1) to increment immediately and return modified object
    StepCounter c2 = ++c1;
    c2.display(); // Prints 10 instead of 11!
    c1.display(); // Prints 11

    return 0;
}
```

Student Tasks

1. Detail the standard C++ signatures and return types for:

- Prefix increment (`++obj`)
- Postfix increment (`obj++`)

2. Explain the purpose of the dummy `int` parameter in postfix operator overloading.

3. Explain why prefix returns by reference (`StepCounter&`) while postfix returns by value (`StepCounter`).

4. Provide the corrected C++ code demonstrating accurate prefix and postfix behavior.

Question 5: Violation of Fundamental Operator Overloading Rules

Topic: Rules for Overloading Operators

Problem Statement

A student submits code attempting custom operator overloading for mathematical vector operations, but violates several core C++ language constraints, triggering multiple fatal compiler rejections.

Buggy Code

```
#include <iostream>

class MathVector {
public:
    double x, y;
    MathVector(double a, double b) : x(a), y(b) {}

    // Line 8: Attempting to overload member access dot operator
    bool operator.(const MathVector &other) {
        return (x == other.x && y == other.y);
    }
};

// Line 14: Attempting to overload operator for built-in primitive types
int operator+(int a, int b) {
    return a * b;
}

// Line 19: Attempting to invent a non-existent operator '**' for exponentiation
MathVector operator**(MathVector a, int power) {
    // ...
}

int main() {
    MathVector v1(2.0, 3.0), v2(2.0, 3.0);
    return 0;
}
```

Student Tasks

1. Enumerate the operators in C++ that **cannot be overloaded under any circumstances**. Why is `operator.` prohibited?

2. State the C++ language rule that causes `int operator+(int, int)` on **Line 14** to be rejected.

3. Can a programmer define new operator tokens such as `**` in C++? Explain the restriction on operator symbols, precedence, and arity.

4. Rewrite the class cleanly, replacing the invalid operators with legal overloaded operators (`==` and `+`) and a member method for exponentiation.

---

4. Section B: Pseudo-Code Writing Questions (5 Problems)

Question 1: Dynamic String Engine with Deep Copy & Overloaded `+`

Topic: Dynamic Constructors, Deep Copy Constructors, Destructors, Binary `+`

Scenario

Design the pseudo-code for an industrial-grade dynamic string class `DynStr`. The class manages dynamic character buffers on the heap, guarantees complete memory deallocation upon destruction, prevents shallow-copy aliasing, and overloads the `+` operator for string concatenation.

Functional Requirements

1. Class Attributes (Private):

- `dataPtr` (POINTER TO CHARACTER)
- `charCount` (INTEGER)

2. Lifecycle Methods:

- `Constructor(sourceString AS STRING)`: Measures length, allocates dynamic buffer of size `charCount + 1`, and copies characters.
- `Copy Constructor(REF other AS DynStr)`: Performs a **Deep Copy** by allocating a fresh character buffer and copying characters from `other.dataPtr`.
- `Destructor()`: Releases heap memory allocated at `dataPtr` and resets pointer to NULL.

3. Overloaded Operator:

- `OPERATOR +(REF rhs AS DynStr) RETURNS DynStr`:
  - Computes sum of lengths.
  - Allocates temporary buffer, copies left operand text, appends right operand text.
  - Constructs and returns a new `DynStr` object holding the concatenated string.

4. Driver:

- Instantiate two strings, concatenate them using `strC <- strA + strB`, and display result.

Student Tasks

1. Write the formal structured pseudo-code for `DynStr` and the driver execution.

2. Provide a dry-run trace showing heap allocations, buffer sizes, and character copies for `strA = "Hello "` and `strB = "World!"`.

Question 2: Complex Number System with Constructor Overloading & Operators (`+`, `*`, `==`)

Topic: Constructor Overloading, Operator Overloading (`+`, `*`, `==`)

Scenario

Design a mathematical class `Complex` supporting complex arithmetic:

- Constructor overloading: Default constructor $(0 + 0i)$, Single-parameter constructor $(r + 0i)$, and Two-parameter constructor $(r + ii)$.
- Overload `+` for addition: $(a + bi) + (c + di) = (a+c) + (b+d)i$.
- Overload `*` for multiplication: $(a + bi) \times (c + di) = (ac - bd) + (ad + bc)i$.
- Overload `==` to verify equality of real and imaginary components.

Functional Requirements

1. **Class `Complex` Attributes**: `real` (REAL), `imag` (REAL).
2. **Overloaded Constructors**:
- Zero arguments: sets `real = 0.0`, `imag = 0.0`.
- One argument `r`: sets `real = r`, `imag = 0.0`.
- Two arguments `r, i`: sets `real = r`, `imag = i`.
3. **Overloaded Operators**:
- `OPERATOR +(REF other AS Complex) RETURNS Complex`.
- `OPERATOR *(REF other AS Complex) RETURNS Complex`.
- `OPERATOR ==(REF other AS Complex) RETURNS BOOLEAN`.
4. **Display Method**: Prints complex numbers in formatted notation `$a + bi$`.

Student Tasks

1. Write the structured pseudo-code defining the constructors, overloaded operators, and helper methods.
2. Provide a step-by-step trace calculating the product of $C_1 = 3 + 2i$ and $C_2 = 1 + 4i$.

Question 3: Financial Currency Class with Stream I/O & Unary Negation

Topic: Unary Operator Overloading, Friend Stream Operators (`<<`, `>>`)

Scenario

Develop an enterprise financial currency class `Currency` representing monetary values as discrete dollars and cents, with auto-normalization, unary negation, and formatted stream I/O.

Functional Requirements

1. **Class Attributes (Private)**: `dollars` (INTEGER), `cents` (INTEGER).
2. **Constructor**:
- Accepts `(d AS INTEGER = 0, c AS INTEGER = 0)`.
- Automatically normalizes cents ≥ 100 into dollars (e.g., 2 dollars and 150 cents $\rightarrow$ 3 dollars and 50 cents). Handles negative cents appropriately.
3. **Unary Operator `-`**:
- Inverts financial balance: positive balance becomes debt, negative balance becomes credit.
- `OPERATOR -() RETURNS Currency`.
4. **Friend Stream Operators**:
- `FRIEND OPERATOR <<(REF outStream, REF cur AS Currency)`: Formats output as `"$D.CC"` (ensuring two-digit cents formatting).
- `FRIEND OPERATOR >>(REF inStream, REF cur AS Currency)`: Reads dollars and cents, then normalizes.

Student Tasks

1. Write the structured pseudo-code for `Currency` and driver program.
2. Trace an input of `d = 5, c = 275`, its normalization, and the result of applying unary negation `-cur`.

Question 4: Polynomial Expression with Overloaded Subscript `[]` & Destructor

Topic: Dynamic Constructors, Destructors, Overloaded Subscript Operator `[]`

Scenario

Design the pseudo-code for a dynamic `Polynomial` class representing algebraic polynomials of degree N. The class must support dynamic heap array allocation, index-based coefficient access/modification via the subscript operator `[]`, deep copying, and polynomial addition.

Functional Requirements

1. Class Attributes (Private):

- `degree` (INTEGER)
- `coeffArray` (POINTER TO REAL) - Dynamic array of size `degree + 1`.

2. Lifecycle Methods:

- `Constructor(deg AS INTEGER)`: Allocates dynamic array and initializes coefficients to 0.0.
- `Copy Constructor`: Performs deep copy of coefficients into fresh heap array.
- `Destructor`: Deallocates dynamic coefficient buffer.

3. Overloaded Subscript Operator `[]`:

- `OPERATOR [](index AS INTEGER) RETURNS REF REAL`:
  - Bounds check: $0 \le \text{index} \le \text{degree}$.
  - Returns an lvalue reference allowing both reading and assignment (`poly[2] <- 5.0`).

4. Overloaded Binary Operator `+`:

- Sums coefficients of matching powers for two polynomials of potentially differing degrees.

Student Tasks

1. Write the formal structured pseudo-code for `Polynomial`.

2. Trace the addition of $P_1(x) = 3x^2 + 2x + 5$ and $P_2(x) = 4x^3 + x^2 + 7$.

Question 5: 2D Vector with Prefix/Postfix Increment & Compound Assignment

Topic: Unary Operators (Prefix vs Postfix `++`), Compound Assignment (`+=`)

Scenario

Design a Cartesian coordinate vector class `Vector2D` that implements prefix increment, postfix increment, and compound assignment operators according to canonical C++ operator semantics.

Functional Requirements

1. Class Attributes (Private): `x` (REAL), `y` (REAL).

2. Constructors: Parameterized constructor with defaults $(0.0, 0.0)$.

3. Overloaded Operators:

- **Prefix Increment (`++v`)**:
  - Increments both x and y by 1.0.
  - Returns **reference** to current object (`REF THIS`).
- **Postfix Increment (`v++`)**:
  - Accepts a dummy integer token to distinguish its signature.

    - Captures a temporary copy of the object's current state.
    - Increments internal x and y by 1.0.
    - Returns the **unmodified snapshot copy by value**.
- **Compound Assignment (`+=`)**:
    - Adds coordinates of right-hand vector into `THIS`.
    - Returns reference to `THIS`.

Student Tasks

1. Write the structured pseudo-code for `Vector2D` and a driver program.

2. Trace the values of `v1`, `v2`, and `v3` after the execution of:

- `v2 <- ++v1`
- `v3 <- v1++`

(Assume `v1` begins at $(5.0, 8.0)$).

SECTION A – DEBUGGING QUESTIONS

Question 1: Copy Constructor Pass-by-Value Infinite Recursion Trap

- Line 17 (copy-constructor declaration) – Compilation error: a copy constructor must take its source object by reference, normally const DynamicString&. Passing DynamicString by value would require copying the argument to initialize the parameter, which would call the same copy constructor again. The parameter cannot be initialized without recursively invoking itself.
- Use const to permit copying from const objects and temporaries where applicable, and & to avoid creating another object copy merely to pass the source.

```
#include <iostream>
#include <cstring>

class DynamicString {
private:
    char* buffer;
    int length;

public:
    DynamicString(const char* str) {
        length = static_cast<int>(std::strlen(str));
        buffer = new char[length + 1];
        std::strcpy(buffer, str);
    }

    // Correct deep-copy constructor
    DynamicString(const DynamicString& source) {
        length = source.length;
        buffer = new char[length + 1];
        std::strcpy(buffer, source.buffer);
    }

    ~DynamicString() {
        delete[] buffer;
    }

    void print() const {
        std::cout << buffer << std::endl;
    }
};

int main() {
    DynamicString s1("Antigravity C++");
    DynamicString s2 = s1;
    s2.print();
    return 0;
}
```

Dry run: s1 owns a buffer containing “Antigravity C++”. Initializing s2 calls the copy constructor, which allocates a separate buffer and copies the characters. Both objects later release their own buffers.

```
Antigravity C++
```

Question 2: Shallow Copy and Double-Free Runtime Crash

- The compiler-synthesized copy constructor performs member-wise copying. For a raw pointer member, it copies the pointer value (address), not the heap array. The original and copied AudioBuffer therefore point to the same samples array.
- When processAudio receives its argument by value, the copied parameter is destroyed when the function returns and deletes the shared array. The original track then contains a dangling pointer. At main exit, track attempts to delete the same allocation again, causing double-free/heap corruption. Reading track.samples[0] after the first deletion is also undefined behavior.
- Correct fix: define a deep-copy constructor that allocates a new array and copies each element. The two objects then own independent allocations.

```
#include <iostream>

class AudioBuffer {
public:
    int* samples;
    int capacity;

    AudioBuffer(int cap) : capacity(cap) {
        samples = new int[capacity];
        for (int i = 0; i < capacity; ++i)
            samples[i] = i * 10;
        std::cout << "Buffer allocated at: "
                  << static_cast<void*>(samples) << std::endl;
    }

    AudioBuffer(const AudioBuffer& other)
        : capacity(other.capacity) {
        samples = new int[capacity];
        for (int i = 0; i < capacity; ++i)
            samples[i] = other.samples[i];
        std::cout << "Deep-copy allocated at: "
                  << static_cast<void*>(samples) << std::endl;
    }

    ~AudioBuffer() {
        std::cout << "Deallocating buffer at: "
                  << static_cast<void*>(samples) << std::endl;
        delete[] samples;
    }
};

void processAudio(AudioBuffer buf) {
    std::cout << "Processing sample 0: "
              << buf.samples[0] << std::endl;
}

int main() {
    AudioBuffer track(5);
    processAudio(track);
    std::cout << "Back in main, sample 0: "
              << track.samples[0] << std::endl;
    return 0;
}
```

Dry run / output interpretation:

Stage	Object	Ownership and sample 0
Construction	track	Allocates array A; sample[0] = 0.
Pass by value	buf	Copy constructor allocates independent array B and copies all five values.
Inside function	buf	Prints sample 0 = 0.
Function return	buf	Destructor releases array B only.
Back in main	track	Array A remains valid; sample 0 = 0.
Program exit	track	Destructor releases array A only.

```
Buffer allocated at: <address A>
Deep-copy allocated at: <different address B>
Processing sample 0: 0
Deallocating buffer at: <address B>
Back in main, sample 0: 0
Deallocating buffer at: <address A>
```

The two printed addresses must be different, but their actual hexadecimal values depend on the runtime and allocator. Run the program to capture the real addresses for submission.

Question 3: Stream Insertion Operator << Member Function Violation

- Line 11 – Design/overload-placement defect: a binary member operator call left @ right is interpreted as left.operator@(right). Thus std::cout << f1 searches for an operator<< member of std::ostream accepting Fraction; it cannot find Fraction's member operator.
- f1 << std::cout compiles because it translates to f1.operator<<(std::cout), which matches the member signature. However, it reverses the conventional stream syntax and cannot support the expected std::cout << f1 form.
- Correct approach: implement operator<< as a non-member friend taking std::ostream& first and const Fraction& second.

```
#include <iostream>

class Fraction {
private:
    int numerator;
    int denominator;

public:
    Fraction(int n, int d) : numerator(n), denominator(d) {}

    friend std::ostream& operator<<(std::ostream& os,
                                    const Fraction& fraction) {
        os << fraction.numerator << "/" << fraction.denominator;
        return os;
    }
};

int main() {
    Fraction f1(3, 4);
    std::cout << f1 << std::endl;
    return 0;
}
```

Dry run: operator<< receives std::cout as os and f1 as fraction, prints its numerator and denominator, and returns the stream reference so stream operations can be chained.

```
3/4
```

Question 4: Prefix vs. Postfix Increment Signature Inversion

- Prefix signature: StepCounter& operator++(). It takes no dummy argument, increments the current object, and returns *this by reference to support chaining without copying.
- Postfix signature: StepCounter operator++(int). The unused int parameter distinguishes postfix from prefix. It stores the old state, increments the object, and returns the old snapshot by value.

```
#include <iostream>

class StepCounter {
private:
    int steps;

public:
    StepCounter(int s = 0) : steps(s) {}

    // Prefix: increment first, return updated object by reference.
    StepCounter& operator++() {
        ++steps;
        return *this;
    }

    // Postfix: preserve old state, increment, return old copy.
    StepCounter operator++(int) {
        StepCounter old(*this);
        ++steps;
        return old;
    }

    void display() const {
        std::cout << "Steps: " << steps << std::endl;
    }
};

int main() {
    StepCounter c1(10);

    StepCounter c2 = ++c1;
    c2.display();
    c1.display();

    StepCounter c3 = c1++;
    c3.display();
    c1.display();

    return 0;
}
```

Dry run:

Operation	c1 after operation	Assigned object	Explanation
Initial	10	—	Starting value.

c2 = ++c1	11	c2 = 11	Prefix increments first and returns updated object.
c3 = c1++	12	c3 = 11	Postfix returns old snapshot (11), then increments c1.

```
Steps: 11
Steps: 11
Steps: 11
Steps: 12
```

Question 5: Violation of Fundamental Operator Overloading Rules

- Line 8 – operator. is illegal. The member access dot operator is not overloadable because it is part of the language's fundamental member-access syntax.
- Line 14 – int operator+(int, int) is illegal because an overloaded operator must have at least one operand of a class or enumeration type. Both operands here are built-in int values.
- Line 19 – C++ does not allow inventing a new operator token such as **. Programmers may overload only existing operator tokens. Overloading cannot change an operator's precedence, associativity, or number of operands.
- Operators that cannot be overloaded include . , .* , :: , ?: , and sizeof. (The assignment also lists these as the non-overloadable operators.)

```
#include <iostream>

class MathVector {
public:
    double x, y;

    MathVector(double a = 0.0, double b = 0.0) : x(a), y(b) {}

    bool operator==(const MathVector& other) const {
        return x == other.x && y == other.y;
    }

    MathVector operator+(const MathVector& other) const {
        return MathVector(x + other.x, y + other.y);
    }

    // Exponentiation is provided as a named member method,
    // not as a newly invented ** operator.
    MathVector power(int exponent) const {
        MathVector result(1.0, 1.0);
        for (int i = 0; i < exponent; ++i) {
            result.x *= x;
            result.y *= y;
        }
        return result;
    }

    void display() const {
        std::cout << "(" << x << ", " << y << ")" << std::endl;
    }
};
```

```
int main() {
    MathVector v1(2.0, 3.0), v2(2.0, 3.0);
    MathVector sum = v1 + v2;

    std::cout << "Vectors equal: " << (v1 == v2) << std::endl;
    std::cout << "Sum: ";
    sum.display();

    std::cout << "Power 2: ";
    v1.power(2).display();
    return 0;
}
```

Dry run: v1 == v2 is true; v1 + v2 = (4, 6); v1.power(2) = (4, 9).

```
Vectors equal: 1
Sum: (4, 6)
Power 2: (4, 9)
```

SECTION B – PSEUDO-CODE WRITING QUESTIONS

Question 1: Dynamic String Engine with Deep Copy and Overloaded +

- DynStr owns a dynamically allocated character buffer. Its copy constructor creates independent storage, its destructor releases owned storage, and operator + builds a new concatenated string.

```
CLASS DynStr
    PRIVATE:
        dataPtr: POINTER TO CHARACTER
        charCount: INTEGER

    PUBLIC:
        CONSTRUCTOR DynStr(sourceString: STRING)
            charCount <- LENGTH(sourceString)
            ALLOCATE ARRAY[charCount + 1] AS CHARACTER AT dataPtr
            COPY sourceString CHARACTERS INTO dataPtr
            dataPtr[charCount] <- NULL
        END

        COPY CONSTRUCTOR DynStr(REF other AS DynStr)
            charCount <- other.charCount
            ALLOCATE ARRAY[charCount + 1] AS CHARACTER AT dataPtr
            FOR i <- 0 TO charCount - 1
                dataPtr[i] <- other.dataPtr[i]
            ENDFOR
            dataPtr[charCount] <- NULL
        END

        DESTRUCTOR()
            DEALLOCATE dataPtr
            dataPtr <- NULL
            charCount <- 0
        END

        OPERATOR +(REF rhs AS DynStr) RETURNS DynStr
            DECLARE combined: STRING
            combined <- STRING_FROM(dataPtr) + STRING_FROM(rhs.dataPtr)
            DECLARE result: DynStr
            result <- DynStr(combined)
            RETURN result
        END

        METHOD Display()
            OUTPUT STRING_FROM(dataPtr)
        END
ENDCLASS

BEGIN Main
    DECLARE strA: DynStr <- DynStr("Hello ")
    DECLARE strB: DynStr <- DynStr("World!")
    DECLARE strC: DynStr
    strC <- strA + strB
    CALL strC.Display()
END
```

Dry-run trace: strA has 6 visible characters plus a null terminator; strB has 6 visible characters plus a null terminator. The concatenated string has 12 visible characters plus a null terminator.

Object / step	Text	Allocation size	Action
strA	Hello	7 characters	Allocate buffer A; copy six characters and NULL.
strB	World!	7 characters	Allocate buffer B; copy six characters and NULL.
Concatenation	Hello World!	13 characters	Create combined text and allocate a fresh result buffer C.
strC	Hello World!	13 characters	Owns buffer C independently.
Destruction	—	—	Each object releases only its own buffer.

```
Hello World!
```

Implementation note: the pseudo-code's temporary combined string represents a safe concatenation buffer. In a concrete C++ implementation, ensure ownership transfer or copying is handled so that no temporary buffer leaks and no two objects delete the same allocation.

Question 2: Complex Number System with Constructor Overloading and Operators

- Provide three constructors: no arguments initializes 0 + 0i; one argument initializes a real-only value; two arguments initialize real and imaginary components.
- Addition adds corresponding components. Multiplication uses (a + bi)(c + di) = (ac − bd) + (ad + bc)i. Equality checks both components.

```
CLASS Complex
    PRIVATE:
        real: REAL
        imag: REAL

    PUBLIC:
        CONSTRUCTOR Complex()
            real <- 0.0
            imag <- 0.0
        END

        CONSTRUCTOR Complex(r: REAL)
            real <- r
            imag <- 0.0
        END

        CONSTRUCTOR Complex(r: REAL, i: REAL)
            real <- r
            imag <- i
        END

        OPERATOR +(REF other AS Complex) RETURNS Complex
            RETURN Complex(real + other.real,
                           imag + other.imag)
        END

        OPERATOR *(REF other AS Complex) RETURNS Complex
            DECLARE newReal: REAL
            DECLARE newImag: REAL
            newReal <- (real * other.real) - (imag * other.imag)
            newImag <- (real * other.imag) + (imag * other.real)
            RETURN Complex(newReal, newImag)
```

```
        END

        OPERATOR ==(REF other AS Complex) RETURNS BOOLEAN
            RETURN real = other.real AND imag = other.imag
        END

        METHOD Display()
            OUTPUT real, " + ", imag, "i"
        END
ENDCLASS

BEGIN Main
    DECLARE C1: Complex <- Complex(3.0, 2.0)
    DECLARE C2: Complex <- Complex(1.0, 4.0)
    DECLARE product: Complex
    product <- C1 * C2
    CALL product.Display()
END
```

Step	Formula	Result
Real part	$(3 \times 1) - (2 \times 4)$	$3 - 8 = -5$
Imaginary part	$(3 \times 4) + (2 \times 1)$	$12 + 2 = 14$
Product	Real + Imaginary i	$-5 + 14i$

Final product: $C1 \times C2 = -5 + 14i$.

Question 3: Financial Currency Class with Stream I/O and Unary Negation

- To normalize positive or negative cents consistently, convert the input pair into one signed total-cent amount. Derive dollars using truncation toward zero and cents using the signed remainder. This preserves the total monetary value.

```
CLASS Currency
    PRIVATE:
        dollars: INTEGER
        cents: INTEGER

    PUBLIC:
        CONSTRUCTOR Currency(d: INTEGER = 0, c: INTEGER = 0)
            DECLARE totalCents: INTEGER
            totalCents <- (d * 100) + c
            dollars <- TRUNCATE_TOWARD_ZERO(totalCents / 100)
            cents <- totalCents MOD 100
        END

        OPERATOR -() RETURNS Currency
            DECLARE totalCents: INTEGER
            totalCents <- (dollars * 100) + cents
            totalCents <- 0 - totalCents
            RETURN Currency(totalCents / 100,
                            totalCents MOD 100)
        END

        FRIEND OPERATOR <<(REF outStream, REF cur AS Currency)
            DECLARE totalCents: INTEGER
            DECLARE signText: STRING
            totalCents <- (cur.dollars * 100) + cur.cents
```

```
            IF totalCents < 0 THEN signText <- "-"
            ELSE signText <- ""
            ENDIF
            OUTPUT signText, "$",
                   ABS(totalCents) DIV 100, ".",
                   TWO_DIGIT(ABS(totalCents) MOD 100)
            RETURN outStream
        END

        FRIEND OPERATOR >>(REF inStream, REF cur AS Currency)
            DECLARE d, c: INTEGER
            INPUT d, c
            CALL cur.Normalize(d, c)
            RETURN inStream
        END

        METHOD Normalize(d: INTEGER, c: INTEGER)
            DECLARE totalCents: INTEGER
            totalCents <- d * 100 + c
            dollars <- TRUNCATE_TOWARD_ZERO(totalCents / 100)
            cents <- totalCents MOD 100
        END
ENDCLASS

BEGIN Main
    DECLARE cur: Currency
    cur <- Currency(5, 275)
    OUTPUT cur
    DECLARE debt: Currency
    debt <- -cur
    OUTPUT debt
END
```

Stage	Calculation	Normalized value
Input	d = 5, c = 275	Total = 5 × 100 + 275 = 775 cents
Normalize	775 DIV 100 = 7; remainder 75	$7.75
Unary negation	−775 cents	−$7.75

Expected formatted values: $7.75 and -$7.75. The sign is applied to the complete monetary amount, not independently to dollars and cents.

Question 4: Polynomial Expression with Overloaded Subscript [] and Destructor

- The class allocates degree + 1 REAL coefficients. The copy constructor performs a deep copy. The subscript operator returns REF REAL so an element can be read or assigned. Addition creates a result with the larger degree and treats missing higher coefficients as zero.

```
CLASS Polynomial
    PRIVATE:
        degree: INTEGER
        coeffArray: POINTER TO REAL

    PUBLIC:
        CONSTRUCTOR Polynomial(deg: INTEGER)
            degree <- MAX(0, deg)
            ALLOCATE ARRAY[degree + 1] OF REAL AT coeffArray
            FOR i <- 0 TO degree
                coeffArray[i] <- 0.0
```

```
            ENDFOR
        END

        COPY CONSTRUCTOR Polynomial(REF other AS Polynomial)
            degree <- other.degree
            ALLOCATE ARRAY[degree + 1] OF REAL AT coeffArray
            FOR i <- 0 TO degree
                coeffArray[i] <- other.coeffArray[i]
            ENDFOR
        END

        DESTRUCTOR()
            DEALLOCATE coeffArray
            coeffArray <- NULL
        END

        OPERATOR [](index: INTEGER) RETURNS REF REAL
            IF index < 0 OR index > degree THEN
                OUTPUT "ERROR: Coefficient index out of range"
                RETURN REF errorCoefficient
            ENDIF
            RETURN REF coeffArray[index]
        END

        OPERATOR +(REF other AS Polynomial) RETURNS Polynomial
            DECLARE maxDegree: INTEGER
            DECLARE result: Polynomial
            DECLARE leftValue, rightValue: REAL
            maxDegree <- MAX(degree, other.degree)
            result <- Polynomial(maxDegree)

            FOR i <- 0 TO maxDegree
                leftValue <- 0.0
                rightValue <- 0.0
                IF i <= degree THEN leftValue <- coeffArray[i] ENDIF
                IF i <= other.degree THEN
                    rightValue <- other.coeffArray[i]
                ENDIF
                result.coeffArray[i] <- leftValue + rightValue
            ENDFOR
            RETURN result
        END
ENDCLASS

BEGIN Main
    DECLARE P1: Polynomial <- Polynomial(2)
    DECLARE P2: Polynomial <- Polynomial(3)
    P1[0] <- 5
    P1[1] <- 2
    P1[2] <- 3
    P2[0] <- 7
    P2[1] <- 0
    P2[2] <- 1
    P2[3] <- 4

    DECLARE P3: Polynomial
    P3 <- P1 + P2
```

```
    OUTPUT "P3(x) = 4x^3 + 4x^2 + 2x + 12"
END
```

Coefficient convention: coeffArray[0] stores the constant term, coeffArray[1] stores the x coefficient, and so on.

Power	P1 coefficient	P2 coefficient	Sum coefficient
x^0	5	7	12
x^1	2	0	2
x^2	3	1	4
x^3	0	4	4

Therefore, P1(x) + P2(x) = $4x^3 + 4x^2 + 2x + 12$.

Safety note: the pseudo-code references errorCoefficient for an invalid index; it should be a persistent class-level fallback value or the operation should terminate with an error. It must never return a reference to a local temporary.

Question 5: 2D Vector with Prefix/Postfix Increment and Compound Assignment

- Prefix ++ increments x and y and returns a reference to the current object. Postfix ++ saves a copy of the old state, increments the current object, and returns the old copy by value. += adds corresponding coordinates and returns a reference to the current object.

```
CLASS Vector2D
    PRIVATE:
        x: REAL
        y: REAL

    PUBLIC:
        CONSTRUCTOR Vector2D(a: REAL = 0.0, b: REAL = 0.0)
            x <- a
            y <- b
        END

        OPERATOR ++() RETURNS REF Vector2D
            x <- x + 1.0
            y <- y + 1.0
            RETURN REF THIS
        END

        OPERATOR ++(dummy: INTEGER) RETURNS Vector2D
            DECLARE oldState: Vector2D
            oldState <- COPY OF THIS
            x <- x + 1.0
            y <- y + 1.0
            RETURN oldState
        END

        OPERATOR +=(REF rhs AS Vector2D) RETURNS REF Vector2D
            x <- x + rhs.x
            y <- y + rhs.y
            RETURN REF THIS
        END

        METHOD Display()
            OUTPUT "(", x, ", ", y, ")"
```

```
        END
ENDCLASS

BEGIN Main
    DECLARE v1: Vector2D <- Vector2D(5.0, 8.0)
    DECLARE v2, v3: Vector2D

    v2 <- ++v1
    v3 <- v1++

    OUTPUT "v1: "
    CALL v1.Display()
    OUTPUT "v2: "
    CALL v2.Display()
    OUTPUT "v3: "
    CALL v3.Display()
END
```

Step	Operation	v1	v2	v3
Initial	—	(5.0, 8.0)	Not assigned	Not assigned
1	v2 <- ++v1	(6.0, 9.0)	(6.0, 9.0)	Not assigned
2	v3 <- v1++	(7.0, 10.0)	(6.0, 9.0)	(6.0, 9.0)

Final values: v1 = (7.0, 10.0), v2 = (6.0, 9.0), and v3 = (6.0, 9.0).